

ISM3232 — Week 7 Lab

Functions, Modules & pytest — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 07 · Unit 2 · Python Foundations

Contents

1	Session Snapshot	2
2	Learning Objectives	2
3	Before Class — Setup Checklist	3
4	Materials Needed	3
5	Timing Plan (75 minutes)	3
6	Segment-by-Segment Walkthrough	4
6.1	Intro (0:00–0:03)	4
6.2	Part 1 — Required Project Structure (0:03–0:12, 9 min)	4
6.3	Part 2 — Write the Functions (0:12–0:28, 16 min)	6
6.4	Part 3 — Import and Call in <code>main.py</code> (0:28–0:38, 10 min)	9
6.5	Part 4 — Eight pytest Tests (0:38–0:58, 20 min)	11
6.6	Part 5 — Ritual and Push (0:58–1:06, 8 min)	14
6.7	Stretch — Doctest (1:06–1:15, as time allows)	15
7	Wrap-Up (last ~9 minutes)	16
8	Appendix A — Full Answer Key (<code>business_rules.py</code> + <code>main.py</code> + <code>tests/test_business_rules.py</code>)	16
9	Appendix B — Extra Practice (only if the class finishes early)	18

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 7 Lab — Functions, Modules & pytest
Unit	Unit 2 · Python Foundations
Class length	Full class period (75 minutes)
Format	Live code-along, ending with the full submission ritual
Prerequisites	Week 6: list-of-dicts, conditionals, loops, pytest basics (marked Midterm-Eligible)
Student-facing lab page	Week 7 In-Class Lab — Module 5, “Functions, Modules, and pytest”
Parts covered	Part 1 (project structure) – Part 5 (ritual + push) + Stretch (doctest)
Submission	Git commit (message must include “lab 7”) + Canvas URL, completion credit

This is the first lab this semester with a genuine **multi-file project structure** — a separate `business_rules.py` module, imported by `main.py`, tested by a third file in `tests/`. The lab page’s own warning deserves to be read verbatim and taken seriously: **no `print()` at the module level in `business_rules.py`** — functions return values; `main.py` prints them; tests assert them. This is a real, professional separation of concerns (business logic vs. presentation vs. verification), not an arbitrary rule, and it’s worth explaining *why* as much as enforcing *that*. This week’s provided code — both `business_rules.py` and all eight tests — is correct as written; no hidden bugs this time, verified directly against the actual HTML source (Week 5’s lab page had a text-rendering issue in this guide’s own source material, not the course’s — worth mentioning only if directly relevant to your own prep).

2 Learning Objectives

By the end of this class period, students should be able to:

1. Organize a project into a business-logic module (`business_rules.py`), a runnable entry point (`main.py`), and a test file — and explain why keeping logic and presentation separate matters.
2. Write functions with docstrings, type hints on parameters and return values, and a return statement — not a `print()` — as the function’s actual output.
3. Import specific functions from a local module with `from business_rules import ...`.
4. Write **boundary-case tests** — testing the exact edge where a rule’s behavior changes, not just a clearly-true and clearly-false case.
5. Add a doctest — a runnable example embedded directly in a docstring.

3 Before Class — Setup Checklist

- ☐ Rehearse the full `business_rules.py` → `main.py` → `tests/test_business_rules.py` sequence once before class — this lab has more interdependent files than any prior week, and confirming the import chain works end-to-end on your own machine first avoids live surprises.
- ☐ Decide how explicitly you'll enforce the “no `print()` in `business_rules.py`” rule — consider actually adding a stray `print()` to a function live, running `main.py`, and asking the room what looks wrong about the output (a debug-looking line appearing outside `main.py`'s own controlled formatting) — a concrete demonstration lands better than a rule stated in the abstract.
- ☐ This lab's provided code (both `business_rules.py` and the eight tests) passes correctly as written when copied verbatim, verified directly against the source lab page — no diagnostic detour needed this week, unlike Week 5.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+
- Students: a fresh `~/ism3232/module05_functions/` project folder

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:03	Welcome: “logic, presentation, verification — three separate files”	3
0:03–0:12	Part 1 — Required project structure	9
0:12–0:28	Part 2 — Write the functions	16
0:28–0:38	Part 3 — Import and call in <code>main.py</code>	10
0:38–0:58	Part 4 — Eight pytest tests	20
0:58–1:06	Part 5 — Ritual and push	8
1:06–1:15	Stretch (doctest) + wrap-up	9

Part 4 gets the most time of any single part, since **boundary-case testing** is a genuinely new, important idea this lab introduces and deserves unhurried treatment, not just eight tests typed and run quickly.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:03)

Say to the class:

“Three files today, each with one job: `business_rules.py` computes things and returns values — nothing else. `main.py` calls those functions and prints the results — it’s the only file that talks to a human. `tests/test_business_rules.py` calls the same functions and checks the results are correct — it never prints anything either, `pytest` reports pass/fail on its own. This separation — logic, presentation, verification, each in its own file — is how real software projects are organized, not a classroom simplification.”

6.2 Part 1 — Required Project Structure (0:03–0:12, 9 min)

Teaching goal: A fresh project setup, now creating three files up front (`business_rules.py`, `main.py`, `tests/test_business_rules.py`) before writing any content into them.

Say to the class:

“Same setup ritual as every module — but notice we’re creating all three files now, empty, before writing a single line into any of them. This is deliberate: seeing the whole structure laid out first makes the ‘who does what’ separation concrete before the details fill in.”

Live-code this:

```
cd ~/ism3232/module05_functions
python3 -m venv .venv && source .venv/bin/activate
pip install pytest ruff && pip freeze > requirements.txt
echo '.venv/' > .gitignore && echo '__pycache__/' >> .gitignore
touch business_rules.py main.py
mkdir -p tests && touch tests/__init__.py tests/test_business_rules.py
```

Line-by-line explanation: The first four lines are Week 3/5’s now-automatic setup sequence, unchanged. The final two lines are new specifically in what they create: `business_rules.py` and `main.py` side by side at the project root, plus `tests/test_business_rules.py` nested one level deeper — say explicitly, pointing at the structure: **this three-file layout, more than any single line of code today, is the actual lesson of Part 1.**

Common student mistakes to watch for:

- Creating `test_business_rules.py` directly in the project root instead of inside `tests/` — `pytest` can often still discover it there, but it breaks the organizational convention established since Week 4 and makes the project harder to navigate as it grows; worth a quick structural check before moving on.

Check for understanding: “Before writing any code, what do you predict `main.py` needs to contain a line of, near its top, that today’s other lab files haven’t needed?” (An `import` statement, pulling in the functions from `business_rules.py` — a good prediction check that primes Part 3’s actual content.)

6.3 Part 2 — Write the Functions (0:12–0:28, 16 min)

Teaching goal: Four business-rule functions, each with a docstring, type hints, and a return — genuinely new syntax (type hints) combined with the “functions return, they don’t print” discipline stated up front.

Say to the class:

“Four functions. Every single one needs three things: a docstring explaining what it does, type hints on its parameters and return value, and a return statement — never a print(). Watch for that last rule specifically as we go.”

Live-code this:

```
# business_rules.py
# Author: [Your Name]

APPROVAL_LIMIT = 1000

def calculate_total(price: float, quantity: int) -> float:
    """Return total cost including 7% tax."""
    return price * quantity * 1.07

def requires_review(amount: float) -> bool:
    """Return True if amount exceeds the approval limit."""
    return amount > APPROVAL_LIMIT

def get_approval_tier(amount: float) -> str:
    """Return the approval routing tier."""
    if amount <= 500: return 'auto'
    elif amount <= 2000: return 'manager'
    else: return 'director'

def apply_discount(price: float, pct: float) -> float:
    """Return price after discount. pct is 0-100."""
    return price * (1 - pct / 100)
```

Line-by-line explanation:

- `APPROVAL_LIMIT = 1000` — Week 6’s named-constant convention, now living at the **module level**, above any function — say explicitly: this makes `APPROVAL_LIMIT` available to every function in this file that needs it (here, just `requires_review`), without needing to pass it as a parameter every time — appropriate for a value that’s genuinely fixed across the whole module, as opposed to something that varies per call.

- `def calculate_total(price: float, quantity: int) -> float:` — **type hints, new syntax today**. `price: float` and `quantity: int` after each parameter name are **annotations** stating the *expected* type of each argument; `-> float` after the closing parenthesis states the *expected* return type. Say explicitly, since this is worth being precise about: **Python does not enforce these at runtime** — calling `calculate_total("bad", "input")` wouldn't raise an error just from the hints alone; type hints are documentation, primarily for humans and for external tools (linters, IDEs) that can check them separately. They're worth using anyway, since they make a function's contract immediately readable without opening its body.
- `"""Return total cost including 7% tax."""` — a **docstring**: a string literal immediately following the `def` line, describing what the function does. Say explicitly: this is retrievable later with `help(calculate_total)` or shown automatically by most editors' hover tooltips — genuinely useful documentation, not a comment that happens to use triple quotes.
- `return price * quantity * 1.07` — **the entire function body is one line, and it's a return, never a print()**. This is the rule stated in the intro, now concretely visible: say explicitly, "if I wanted to see this value, I do that in `main.py` — this function's only job is to compute and hand back a number."
- `def get_approval_tier(amount: float) -> str:` — the richest function today, using an `if/elif/else` chain to classify an amount into one of three string tiers. Note the **single-line if/elif/else bodies** (`if amount <= 500: return 'auto'`, all on one line) — say explicitly this is a valid but relatively unusual style choice (most code would put the `return` on its own indented line below) — it's shown compactly here specifically because each branch is a single, simple statement; worth mentioning both styles are equivalent, and a student's own preference for the more conventional multi-line form is completely fine.
- `def apply_discount(price: float, pct: float) -> float:` — `pct` is documented (via the docstring, not a type hint, since "0 to 100" isn't expressible as a Python type) as a percentage on a 0–100 scale, not a 0–1 decimal — say explicitly why this convention matters: `apply_discount(100.00, 10)` means "10%", computed as `price * (1 - pct/100)`, i.e., `100 * (1 - 0.10) = 90.0` — a student who assumed `pct` should be passed as `0.10` directly would get a wildly wrong (and non-obviously wrong) result, since `100 * (1 - 0.10/100)` computes something very different. This is exactly the kind of ambiguity a docstring exists to resolve.

Common student mistakes to watch for:

- Adding a `print()` inside one of these functions "just to check it's working" while developing, and forgetting to remove it — walk the room specifically checking for this, since it's a very natural debugging instinct that directly violates this lab's stated rule; a good moment to suggest an alternative: temporarily calling the function from the Python REPL (a separate, throwaway check) instead of embedding a `print` inside the function itself.
- Confusing the boundary logic in `get_approval_tier` — note explicitly that `<=500` and `<=2000` use **inclusive** boundaries (worth connecting directly to Part 4's dedicated boundary-case tests, which exist specifically to verify this).

- Forgetting type hints entirely, or applying them inconsistently (hints on some parameters but not others) — not a runtime error, but worth holding the room to the lab’s explicit “every function must have” standard.

Check for understanding: “Without running it, what does `get_approval_tier(500)` return — exactly at the first boundary?” (`'auto'` — since the check is `<= 500`, inclusive; a good direct preview of Part 4’s `test_tier_boundary_500`.)

6.4 Part 3 — Import and Call in main.py (0:28–0:38, 10 min)

Teaching goal: Import specific functions from a local module, and confirm all four work together — main.py’s entire, deliberately narrow job.

Say to the class:

“This file has exactly one job: call the four functions from business_rules.py and print what they return. No business logic lives here — if you find yourself computing something in main.py beyond formatting a print statement, that computation probably belongs in business_rules.py instead.”

Live-code this:

```
# main.py
from business_rules import calculate_total, requires_review, get_approval_tier, apply_discount

price, qty = 450.00, 3
total = calculate_total(price, qty)

print(f'Total:          ${total:.2f}')
print(f'Requires review: {requires_review(total)}')
print(f'Approval tier:   {get_approval_tier(total)}')
print(f'10% discount:    ${apply_discount(price, 10):.2f}')
```

Line-by-line explanation:

- from business_rules import calculate_total, requires_review, get_approval_tier, apply_discount — **new import style today**. Say explicitly, contrasting with import pandas as pd-style imports from other contexts (if relevant to your section): business_rules here is not an installed package — it’s business_rules.py, a file **sitting right next to main.py in the same folder**. Python automatically looks in the current file’s own directory for a module by that name, no installation required. from ... import name1, name2, ... pulls in specific functions by name, so they can be called directly (calculate_total(...)) rather than needing a prefix (business_rules.calculate_total(...), which would be the result of a plain import business_rules instead).
- price, qty = 450.00, 3 — tuple unpacking, familiar from prior modules, setting up the two values calculate_total needs.
- total = calculate_total(price, qty) — the function call, storing the returned value — say explicitly, this is the first concrete payoff of “functions return values”: total now holds a real number, usable in every subsequent line, which would be impossible if calculate_total had only print()ed instead.
- The four print() calls — each one calling a function and immediately formatting its result — say explicitly, this is **exactly and only** what main.py is for: no computation happens directly in this file, every actual business calculation is delegated to business_rules.py.

Run it. Verified output:

```
Total:          $1444.50
Requires review: True
Approval tier:   manager
10% discount:   $405.00
```

Common student mistakes to watch for:

- Running `python3 main.py` from the wrong directory (not `module05_functions/`) — the import fails with `ModuleNotFoundError: No module named 'business_rules'`, since Python's search for a local module starts from the current working directory; a good direct callback to every prior module's location-awareness lessons.
- Misspelling an imported function name, or forgetting one in the `from ... import` list — raises `ImportError` naming the missing function specifically, or `NameError` at the point of use if the import line itself succeeded but a function wasn't actually included in it — worth reading either error type together if it comes up.
- Computing something directly in `main.py` (e.g., manually recomputing tax inline instead of calling `calculate_total`) — not a syntax error, but a direct violation of the separation-of-concerns principle this lab is building; redirect explicitly back to “does this belong in `business_rules.py` instead?”

Check for understanding: “If you wanted to add a fifth business rule tomorrow, which file would you edit first, and would `main.py` need to change at all?” (`business_rules.py` first, to define the new function; `main.py` would only need to change if you also wanted to *display* that new rule's result — the two concerns are genuinely independent, which is the entire point of today's structure.)

6.5 Part 4 — Eight pytest Tests (0:38–0:58, 20 min)

Teaching goal: Write eight tests, including at least two genuine **boundary cases** — testing the exact edge where a rule’s behavior changes, not just an obviously-true and obviously-false case.

Say to the class:

“Eight tests, and I want to spend real time on two of them specifically — the boundary tests. A boundary case tests the exact edge of a rule, where behavior flips. For a threshold of 1000, that’s testing with exactly 1000, not just a comfortably-above or comfortably-below number. This is where the most dangerous real bugs actually hide — an off-by-one in a threshold check, a $>$ where you meant $\geq$.”

Live-code this:

```
# tests/test_business_rules.py
from business_rules import calculate_total, requires_review, get_approval_tier, apply_discount

def test_total_includes_tax():
    assert round(calculate_total(100.00, 2), 2) == 214.00

def test_requires_review_over():
    assert requires_review(1500) is True

def test_requires_review_under():
    assert requires_review(500) is False

def test_boundary_at_limit():
    # Exactly at 1000 should NOT require review (> not >=)
    assert requires_review(1000) is False

def test_tier_auto():
    assert get_approval_tier(400) == 'auto'

def test_tier_boundary_500():
    assert get_approval_tier(500) == 'auto' # exactly at limit

def test_tier_manager():
    assert get_approval_tier(1500) == 'manager'

def test_discount_ten():
    assert apply_discount(100.00, 10) == 90.0
```

Line-by-line explanation:

- `from business_rules import ...` — the exact same import line as `main.py`, worth noting explicitly: **tests import the module under test exactly the same way any other code would** — there's nothing special about how a test file accesses the functions it's checking.
- `test_total_includes_tax` — `round(calculate_total(100.00, 2), 2) == 214.00` — note the `round(..., 2)` wrapping the function call, worth connecting directly back to Week 5's floating-point lesson: $100.00 * 2 * 1.07$ may not land on an exact `214.00` in raw binary floating point, so rounding to 2 decimal places before comparing sidesteps that precision issue entirely — a good, concrete callback to a lesson from two weeks ago, now being *applied* rather than freshly discovered.
- `test_requires_review_over` and `test_requires_review_under` — the two “obviously true, obviously false” cases: `1500` clearly requires review, `500` clearly doesn't. Say explicitly: **these two alone are not sufficient testing** — they confirm the function works for clear-cut inputs, but say nothing about what happens right at the threshold, which is exactly the gap the next test fills.
- `test_boundary_at_limit` — **this is the lab's featured boundary case, and its own comment states exactly why it exists:** `requires_review(1000)` should be `False`, since the function's actual condition is `amount > APPROVAL_LIMIT` (strictly greater than), not `>=`. Say explicitly, slowly: **if a developer had written `>=` instead of `>` by mistake, `test_requires_review_over` and `test_requires_review_under` would both still pass** — neither of them tests the value `1000` itself — **only this boundary test would catch that specific mistake**. This is the single most important idea in this whole lab: comfortable, clearly-true/false tests can give false confidence; boundary tests catch the bugs that actually matter.
- `test_tier_boundary_500` — the second required boundary case, for `get_approval_tier`'s first threshold — same principle, different function: confirming `500` itself (the inclusive boundary) lands in `'auto'`, not `'manager'`.
- `test_discount_ten` — `apply_discount(100.00, 10) == 90.0` — note this uses plain `==`, not `round()` or `math.isclose()` — worth a brief “why does this one not need the floating-point safety net Test 1 used?” discussion: $100.00 * (1 - 10/100)$ happens to compute to an exact `90.0` in this specific case (unlike $200.00 * 0.07$, Week 5's problem case) — not because floating-point issues never apply here, but because this particular arithmetic happens not to trigger them. A good moment to note that floating-point precision issues are input-dependent, not universal — some calculations land exactly, others don't, and it's not always obvious in advance which.

Run it:

```
pytest -v
```

Verified output — all eight pass as provided:

```
tests/test_business_rules.py::test_total_includes_tax PASSED
tests/test_business_rules.py::test_requires_review_over PASSED
tests/test_business_rules.py::test_requires_review_under PASSED
```

```
tests/test_business_rules.py::test_boundary_at_limit PASSED
tests/test_business_rules.py::test_tier_auto PASSED
tests/test_business_rules.py::test_tier_boundary_500 PASSED
tests/test_business_rules.py::test_tier_manager PASSED
tests/test_business_rules.py::test_discount_ten PASSED
8 passed
```

A genuinely worthwhile live demo, if time allows: deliberately change `requires_review's >` to `>=` in `business_rules.py`, save, and re-run `pytest -v`. **`test_boundary_at_limit` fails; every other test still passes.** This is worth doing live specifically to make the abstract “boundary tests catch what other tests miss” claim concretely, visibly true — then revert the change back to `>` before moving on.

Common student mistakes to watch for:

- Treating `requires_review(1500) is True` as risky, chained-comparison-prone syntax, per Week 5's lesson — reassure explicitly: this is safe, since `requires_review(1500)` is a single function call evaluated first, *then* compared with `is True` — the Week 5 hazard was specifically about chaining a comparison operator (`>`) directly against `is` in one unbroken expression (`1500 > 1000 is True`), which this isn't; a good moment to have a student articulate the difference rather than just reassure them.
- Writing only “obviously true/false” tests and skipping genuine boundary cases when adding their own tests later (e.g., in Appendix B's extra practice) — this is exactly the habit this lab exists to correct; hold students to including at least one real boundary case in any test set they write independently from today forward.

Check for understanding: “For `get_approval_tier`'s second threshold, at 2000, what boundary test would you write, and what should it assert?” (`assert get_approval_tier(2000) == 'manager'` — since the condition is `<= 2000`, inclusive; getting a student to construct this test themselves, unprompted, confirms the boundary-testing instinct generalized beyond the two examples explicitly given.)

6.6 Part 5 — Ritual and Push (0:58–1:06, 8 min)

Teaching goal: The established five-step ritual, now including `-v` on the test step by default — since this lab’s tests are numerous and specific enough that seeing each one named is worth the extra output.

Say to the class:

“Same ritual — note `pytest -v` this time instead of bare `pytest`, since with eight tests, seeing each one individually named as it passes is worth the extra verbosity.”

Live-code this:

```
ruff format . && ruff check . && pytest -v  
git add . && git commit -m 'lab 7: functions modules pytest' && git push
```

Common student mistakes to watch for: None new this week — this is pure repetition of an already-established sequence; the main thing worth watching for is students continuing to run it confidently without needing the steps spelled out, confirming the habit from Weeks 4–6 has genuinely stuck.

Check for understanding: “This is the fourth week running this ritual. What’s changed about how you type it, compared to Week 4?” (A good closing, reflective question rather than a technical one — most students should report it’s become noticeably faster and more automatic, which is worth naming explicitly as the whole point of repeating it identically, week after week.)

6.7 Stretch — Doctest (1:06–1:15, as time allows)

Frame as a quick, genuinely useful addition if the room reaches it:

```
def calculate_total(price: float, quantity: int) -> float:
    """
    Return total cost including 7% tax.

    >>> calculate_total(100.0, 2)
    214.0
    """
    return price * quantity * 1.07
```

Run with:

```
python3 -m doctest business_rules.py -v
```

Verified output (abbreviated):

Trying:

```
calculate_total(100.0, 2)
```

Expecting:

```
214.0
```

ok

```
1 tests in business_rules.calculate_total
```

```
1 passed and 0 failed.
```

```
Test passed.
```

One sentence of framing, worth stating even briefly: “A **doctest** is a runnable example living directly inside a docstring — `>>>` mimics what you’d type at a Python prompt, and the line below it is the *exact* expected output. `python3 -m doctest` runs every such example in the file and confirms the real output matches. This is worth knowing exists as a lightweight alternative to a full `pytest` file for simple, illustrative examples — genuinely useful for documentation that stays verified against reality, since a doctest that goes stale (no longer matches actual behavior) fails loudly, unlike a comment that can silently drift out of date.”

7 Wrap-Up (last ~9 minutes)

Review the submission checklist together:

- ☐ Git commit made, with a message including “lab 7”
- ☐ `business_rules.py` contains all four functions, each with a docstring, type hints, and a return — no `print()`
- ☐ `main.py` imports and calls all four functions, printing formatted results
- ☐ `tests/test_business_rules.py` contains all eight tests, including at least two boundary cases, all passing
- ☐ Full ritual run (format, lint, test, commit, push)
- ☐ GitHub repository URL pasted into Canvas

Preview Week 8: “Today’s rule — no `print()` inside business logic — sets up next week directly: debugging skills and AI literacy, where reading a function’s actual behavior (not just trusting it looks right) becomes the central skill, ahead of the midterm the following week.”

8 Appendix A — Full Answer Key (`business_rules.py` + `main.py` + `tests/test_business_rules.py`)

```
# business_rules.py
# Author: [Your Name]

APPROVAL_LIMIT = 1000

def calculate_total(price: float, quantity: int) -> float:
    """Return total cost including 7% tax."""
    return price * quantity * 1.07

def requires_review(amount: float) -> bool:
    """Return True if amount exceeds the approval limit."""
    return amount > APPROVAL_LIMIT

def get_approval_tier(amount: float) -> str:
    """Return the approval routing tier."""
    if amount <= 500:
        return 'auto'
    elif amount <= 2000:
        return 'manager'
    else:
        return 'director'
```

```
def apply_discount(price: float, pct: float) -> float:
    """Return price after discount. pct is 0-100."""
    return price * (1 - pct / 100)
```

main.py

```
from business_rules import calculate_total, requires_review, get_approval_tier, apply_discount
```

```
price, qty = 450.00, 3
```

```
total = calculate_total(price, qty)
```

```
print(f'Total:          ${total:.2f}')
```

```
print(f'Requires review: {requires_review(total)}')
```

```
print(f'Approval tier:   {get_approval_tier(total)}')
```

```
print(f'10% discount:    ${apply_discount(price, 10):.2f}')
```

tests/test_business_rules.py

```
from business_rules import calculate_total, requires_review, get_approval_tier, apply_discount
```

```
def test_total_includes_tax():
```

```
    assert round(calculate_total(100.00, 2), 2) == 214.00
```

```
def test_requires_review_over():
```

```
    assert requires_review(1500) is True
```

```
def test_requires_review_under():
```

```
    assert requires_review(500) is False
```

```
def test_boundary_at_limit():
```

```
    # Exactly at 1000 should NOT require review (> not >=)
```

```
    assert requires_review(1000) is False
```

```
def test_tier_auto():
```

```
    assert get_approval_tier(400) == 'auto'
```

```
def test_tier_boundary_500():
```

```
    assert get_approval_tier(500) == 'auto' # exactly at limit
```

```
def test_tier_manager():
```

```
    assert get_approval_tier(1500) == 'manager'
```

```
def test_discount_ten():
```

```
assert apply_discount(100.00, 10) == 90.0
```

9 Appendix B — Extra Practice (only if the class finishes early)

Four required parts plus the ritual fill the full class period at a normal pace, especially given Part 4's deliberate boundary-testing depth. If a section moves unusually fast:

Extra — the second `get_approval_tier` boundary. Have students add `test_tier_boundary_2000`, asserting `get_approval_tier(2000) == 'manager'` (the check-for-understanding question from Part 4, now actually written and run) plus `test_tier_boundary_2001`, asserting `get_approval_tier(2001) == 'director'` — a complete pair confirming both sides of the second threshold, not just one.

Extra — a fifth function, with its own boundary test. Have students add `def is_bulk_order(quantity: int) -> bool: """Return True if quantity is 10 or more.""" return quantity >= 10` to `business_rules.py`, call it from `main.py`, and write two tests for it: an obvious case and a genuine boundary case at exactly 10 (verified: `is_bulk_order(10)` should be `True`, since the condition is `>=`, inclusive — the opposite boundary direction from `requires_review`'s `>`, worth having students notice and state explicitly why the two functions' boundary behaviors differ).